When should an AI model help plan meals?

Sai Teja Paladi
CSCE798 Independent Study
`https://github.com/TejPaladi/beacon-knapsack-agentic-evaluation`
November 5, 2026

1 The question, in plain words

BEACON is a meal-planning system [1]. Its *Knapsack* method picks a day of meals for a person by following fixed rules, so the same request always gives the same plan. This study asks: **which jobs should an AI model do, and which should stay as ordinary code?** Following *Doing Less with More* [2], the idea is to use the model only where understanding language helps, and to keep exact work, like choosing safe food, in code.

- **AI model:** a program that reads and writes text, like ChatGPT. We use small free models that run on a laptop: Granite 4 (3B), Llama 3.2 (3B and 1B), Qwen 3.5 (4B), and Phi-4 Mini. Experiment 1 uses a larger hosted model (gpt-oss-20b).
- **Agent:** an AI model that can use tools. It reads a request, asks for a tool, reads the result, and repeats until it is done.
- **Tool:** an ordinary program the model may ask to run, such as "look up Becca Moore's profile".
- **Persona:** one of six made-up people with saved diet rules, for example Becca Moore: vegetarian, 2,000 calories a day.
- **Plan check:** BEACON's checker that tests a plan against the person's rules (no meat for a vegetarian, every meal complete, and so on).

All scoring is done by code from what the agent actually did; no AI grades another AI. Every model run uses temperature 0. Where runs were repeated (three times in Experiments 4–7), they gave the same answers.

2 The tools (meeting feedback)

Experiments 1–3 use three tools: `list_personas`, `get_persona`, and `run_knapsack_for_persona`. Following the last meeting, Experiments 4–6 use five tools, one for each job discussed.

Discussed	Final tool	What it does	Why it is needed
PersonaSelector + PersonaRetriever	`fetch_persona_profile`	finds the named person and reads their saved profile	combined: finding a person is always followed by reading the profile, and passing a name instead of an ID caused 41 of the 71 failures on named requests in Experiment 2
Recommender	`recommend_meal_plan`	runs Knapsack to choose the day's meals	the model never chooses food itself

Discussed	Final tool	What it does	Why it is needed
NutritionCalculator	`calculate_nutrition_scores`	totals calories, protein, and health scores	only used when the user asks for numbers
Violation Checker	`check_plan_violations`	runs the plan check	the result decides whether the plan should be shown
(new)	`release_plan`	shows the plan to the user	makes "show it or not" a choice we can observe

A tool counts as **meaningful** if it does one job, some requests need it while others do not, and its result changes what happens next.

3 Research questions and answers

Each expectation was written down before the data it is tested on.

Exp.	Question	What we expected	Result
2	Can small models carry out the steps, and do they ask when it is unclear who the plan is for?	no model does both well	**Yes, as expected**
4	Does the model leave the food choice to Knapsack?	every plan shown is exactly Knapsack's	**Yes**
4	Does the model hold back a plan that failed its own check?	some models show it anyway	**Yes, they do**
4	Does the model understand indirect wording better than keyword code?	yes	**No**
2, 4	Is the model slower than code alone?	yes	**Yes, 17–190 times**
5	With messy names, does the best model find the right person more often than good rules?	yes	**No (too close to call)**
5	… does it also pick the wrong person more often?	yes	**Yes**
6	If the model suggests a person and the user confirms, does the right plan arrive more often than with name matching?	yes, for every model	**For 2 of 3 models**
6	… and does no plan go to the wrong person?	yes	**Yes**
7	Choosing among safe plans, does the model follow a goal ("I'm trying to drop a few pounds") better than keyword code?	yes	**No (too close to call)**
7	… is keyword code as good when the wish is stated directly?	yes	**Yes**
7	… are all chosen plans safe?	yes	**Yes**

"Too close to call" means the difference is small enough to be chance. Experiments 1 and 3 are measurements without an expectation.

4 Experiment 1: Does an agent change Knapsack's plans?

Goal: check that an agent calling the three tools gets exactly the plan that calling Knapsack directly gives.

Example: "Please generate a meal plan for Earl Jones." The agent calls `list_personas`, then `get_persona`, then `run_knapsack_for_persona`, and returns the plan.

Method	Plans completed	Same as Knapsack	BEACON quality	Average time
Knapsack called directly	6/6	–	0.860	0.2 s
Agent (hosted model)	6/6	6/6	0.860	5.7 s

Finding: the agent reproduces every plan exactly, but takes 28 times longer.

5 Experiment 2: 30 requests, five small models

Goal: 24 requests name a person in different ways (full name, first name, paraphrase, "ignore my diet"); 6 are unclear (no name, an unknown name, or two names), where the right answer is to ask. A keyword script with no AI does the same task for comparison.

Example: "Give me a meal plan for today." Granite does not ask who the plan is for; it picks a person from the list and makes their plan.

Method	All (30)	Named (24)	Unclear: asked (6)	Unclear: made a plan	Avg. time
Keyword script (no AI)	30	24	6	0	0.15 s
Granite 4 3B	22	22	0	6	8.5 s
Qwen 3.5 4B	26	22	4	1	22.6 s
Llama 3.2 3B	5	5	0	3	4.6 s
Phi-4 Mini 3.8B	6	0	6	0	2.5 s
Llama 3.2 1B	0	0	0	0	16.2 s

Finding: doing the steps and asking when unsure are different skills. Granite and Qwen complete 22 of 24 named requests but rarely ask; Phi-4 Mini always asks but completes none. In total the models made 10 plans when they should have asked.

Update from the earlier report: replies were first cut off after 160 tokens, which hid Qwen's answers (16/30). All models were re-run with a 1,024-token limit; the other models' results did not change.

6 Experiment 3: What if the model picks the food itself?

Goal: the model chooses nine items (main, side, and drink for three meals) directly from the 124-item catalog, with no Knapsack. BEACON's own checker and quality score judge the result.

Method	Safe plans	Rule breaks	BEACON quality	Repeated items
Knapsack	6/6	0	0.860	0.0
Granite 4 3B	6/6	0	0.679	2.0
Llama 3.2 3B	3/6	4	0.453	0.0

Finding: Granite stays safe but repeats foods; Llama served meat to vegetarians and sugary food to a person with diabetes. Food choice stays with Knapsack.

7 Experiment 4: Five tools and the decision to show a plan

Goal: 36 requests, each naming one person, in six types (show the profile only, plan meals, plan with nutrition numbers, indirect wording, "ignore my diet", and a test plan we deliberately broke). Each was written twice: the original wording, and new wording written after the keyword script was finished.

Example (broken test plan, Granite): "Recommend a plan for Jessica Smith and verify it before release." The plan check reports a problem, yet Granite calls `release_plan` and tells the user: "The meal plan for Jessica Smith has been successfully released. It meets all the dietary and nutritional requirements."

Method	Correct: original	Correct: new wording	Broken plans shown	Avg. time
Keyword script (no AI)	33	28	0 + 0	0.25 s
Granite 4 3B	24	26	6 + 4	11.5 s
Llama 3.2 3B	23	13	0 + 0	9.6 s
Qwen 3.5 4B	23	24	6 + 6	39.2 s

Correct answers out of 36 on each wording; broken plans shown out of 6 on each wording.

Findings: every plan shown, apart from the deliberately broken ones, was exactly Knapsack's plan, so the model never changed the food. Two of three models showed plans that their own check had rejected, and said they were fine. The keyword script was right more often on both wordings and is about 35 to 190 times faster, so on clearly worded requests the model adds no value yet.

8 Experiment 5: Messy names

Goal: 48 short conversations refer to people the way real users do: nicknames ("Jess"), typos ("beca moor"), descriptions ("our client with diabetes"), earlier messages ("now do the other one"), and Telugu–English or Hindi–English ("Becky ki today em tinalo plan cheyyi"). In 12 of them the right answer is to ask, for example "Greg Thompson" when only Greg Taylor exists. We compare simple name matching, stronger hand-written rules (saved before the conversations were written), and the three models.

Method	Hard names found (30)	Asked when it should (12)	Wrong person (48)
Simple name matching	7	8	5
Stronger rules	27	11	3
Granite 4 3B	28	2	12
Llama 3.2 3B	28	0	14
Qwen 3.5 4B	29	3	10

Finding: the models are good at guessing who is meant, but they guess even when they should ask: all three turned "Greg Thompson" into Greg Taylor. A better design is to let the model *suggest* ("Did you mean Becca Moore?") and let the user confirm. With that design no wrong person was ever chosen, and the right person was reached in 34–35 of 36 cases.

9 Experiment 6: Suggest-and-confirm in the full workflow

Goal: the same 48 conversations, but now the chosen person goes through the whole five-tool agent: fetch the profile, run Knapsack, check the plan, and show it. A simulated user says yes only when the suggested person is correct.

Example (Qwen): "Please set up meals for Greg Thompson today." Deciding alone, Qwen produced Greg Taylor's plan and showed it. With suggest-and-confirm it asks "Did you mean Greg Taylor?"; the user says no, and nothing is shown.

Method	Granite 4 3B	Llama 3.2 3B	Qwen 3.5 4B
Simple name matching	13 / 0	3 / 0	13 / 4
Stronger rules	27 / 1	6 / 0	33 / 2
Model decides alone	28 / 5	6 / 1	35 / 9
Model suggests, user confirms	**28 / 0**	**6 / 0**	**35 / 0**

Each cell: right plans delivered (of 36) / plans shown for the wrong person (of 48).

Findings: suggest-and-confirm never showed a plan for the wrong person. It more than doubles the right plans with Granite and Qwen, but not with Llama: Llama was handed the right person 34 times yet usually stopped after reading the profile. The cost is that the user is asked to confirm in 37 to 40 of the 48 conversations.

10 Experiment 7: Picking the best of several safe plans

Goal: Knapsack plans five different days for each person, and all five pass the plan check. Each of 48 requests asks for one thing (more protein, fewer calories, more fiber, or fewer carbs), half said directly ("the plan with the most fiber") and half as a goal ("my blood sugar spikes after big meals"). A choice is right if it is the best of the five for that nutrient.

Example: "My blood sugar spikes after big meals, so I want to keep it steady" means fewer carbs. The keyword script finds no keyword and keeps the default plan; Qwen picks the lowest-carb plan.

Method	All (48)	Said directly (24)	Said as a goal (24)	Unsafe picks
Knapsack's default plan	16	8	8	0
Keyword script	39	24	15	0
Granite 4 3B	31	19	12	0
Llama 3.2 3B	23	17	6	0
Qwen 3.5 4B	42	24	18	0

Findings: no choice was ever unsafe, because every option was already safe. Qwen reads goals somewhat better than keywords (18 vs. 15), but the difference is too close to call. Granite and Llama even misread which number was smallest. The keyword script and Qwen fail on different requests: using the script when it finds a keyword and Qwen otherwise reached 44 of 48 (an idea found after the run, to be tested properly next).

11 Agentic AI Test Checklist (meeting feedback)

All 110 items of the checklist [3] were reviewed against this system. It is a design aid, not a meal benchmark, so no checklist score is claimed: 22 items are measured, 4 measured as an extra, 15

partly covered, 15 left for future work, and 54 do not apply (one agent, no logins, no outside documents). Examples measured here: tools called in a sensible order, the plan checked before it is shown, and identical answers on repeated runs.

12 What we learned

1. Small AI models can use tools, but they often do not ask when a request is unclear, and two of three showed plans that failed their own check while saying the plans were fine.
2. On clearly worded requests, simple keyword code with no AI was as accurate or better, and 17–190 times faster.
3. The models help with messy wording, but they guess instead of asking. Letting the model suggest and the user confirm keeps the help without the wrong-person plans.
4. The model should not pick food from the whole catalog (Experiment 3), but picking among Knapsack's safe plans is safe and reads goals reasonably well (Experiment 7).

In one sentence: let code choose the food and check it; use the AI model for language, and let it suggest rather than decide.

Limitations: six made-up people; small models on one laptop; the test requests were written by me; the confirming user in Experiment 6 is simulated and never accepts a wrong suggestion, so its result is a best case.

Next steps: real users confirming suggestions; a version where the model only turns a goal into a nutrient ("backed up" means more fiber) and code picks the plan; and larger models.

References

[1] V. Nagpal, S. L. Valluru, K. Lakkaraju, and B. Srivastava. BEACON: Balancing convenience and nutrition in meals with long-term group recommendations and reasoning on multimodal recipes. *arXiv:2406.13714*, 2024.

[2] R. Garimella, B. Srivastava, and A. Sheth. Doing less with more: A first-principles exploration of the suitability of agentic computing over alternative architectural choices. *arXiv:2610.23921*, 2026.

[3] S. Padmanabhuni. Agentic AI Test Checklist 110. Hugging Face dataset, 2026.